

Full Stack Mastery

Complete Assignment Roadmap

10 Layers · Every Topic · Easy / Medium / Hard Assignments

LEARNING TIMELINES

NORMAL PACE (12 months)

Month 1	JavaScript Deep Dive + TypeScript
Month 2	Node.js Internals + Express + Auth
Month 3	PostgreSQL + Redis + Database Design
Month 4	React Deep Dive + State + Data Fetching
Month 5	Next.js (App Router fully)
Month 6	Docker + GitHub Actions + Linux + Nginx
Month 7	Security + Testing
Month 8	Build Ambitious Project 1
Month 9	Build Ambitious Project 2
Month 10	System Design (Alex Xu Vol 1) + DSA
Month 11	Build Generic Project + Polish + Deploy
Month 12	Interview Prep + Applying + Refining Portfolio

FULL-TIME OBSESSIVE PACE (8 months)

Week 1-2	JavaScript Deep Dive
Week 3-4	TypeScript
Month 2	Node.js Internals + Express + Auth
Month 3	PostgreSQL + Redis + Database Design
Month 4	React + State + Data Fetching + Next.js
Month 5	DevOps — Docker, CI/CD, Linux, Nginx, Cloud
Month 5.5	Security + Testing
Month 6	Build Ambitious Project 1
Month 7	Build Ambitious Project 2 + System Design
Month 7.5	DSA — LeetCode Easy/Medium
Month 8	Generic Project + Polish + Interview Prep
Month 8+	Applying — international remote roles

Treat each assignment as a real deliverable. Run the code. Break it. Fix it. That is the curriculum.

LAYER 1

The foundation everything else runs on. Do not rush this layer.

■ The Event Loop — Call Stack, Task Queue, Microtask Queue

EASY

Write a script with a `setTimeout(fn, 0)`, a resolved `Promise.then()`, and a synchronous `console.log()`. Before running it, write down on paper the exact order you expect each to print. Run it. Explain why.

MEDIUM

Build a function called `asyncScheduler(tasks)` that accepts an array of mixed sync functions, Promise-returning functions, and `setTimeout`-wrapped functions. Execute them and log the order they resolve. Then draw the event loop state at each tick.

HARD

Implement a custom microtask queue from scratch using only synchronous JS (no Promise, no `queueMicrotask`). Simulate how the JS engine drains microtasks before moving to the next macrotask. Test it with at least 5 interleaved macro and micro tasks and verify output order manually.

Note: The classic mistake: putting `await` inside a `forEach` loop. Reproduce the bug, explain why it fails, then fix it.

■ Closures, Scope, and Lexical Environment

EASY

Write a `makeCounter()` factory function that returns `{ increment, decrement, value }`. Each call to `makeCounter()` must have its own independent count. Write 3 separate counter instances and prove they do not share state.

MEDIUM

Implement a `memoize(fn)` higher-order function using closures. It must cache results by serialised arguments, handle functions with multiple arguments, and have a `.clear()` method to wipe the cache. Test with a slow Fibonacci function — first call should be slow, second should be instant.

HARD

Build an event system (`on`, `off`, `emit`, `once`) entirely using closures — no classes, no `this`. `once()` listeners must auto-remove after firing once. Demonstrate memory implications: show that holding references inside closures prevents GC, then fix the leak with explicit cleanup.

Note: Classic interview trap: the `var-in-loop` closure bug. Reproduce it with `var`, explain the lexical environment reason it fails, fix it three ways (`let`, `IIFE`, `closure factory`).

■ Prototypes and Prototype Chain

EASY

Without using class, create an `Animal` constructor function with a `speak()` method on its prototype. Create a `Dog` that inherits from `Animal` using `Object.create()`. Verify the chain with `instanceof` and `Object.getPrototypeOf()`.

MEDIUM

Implement your own version of `Object.create()` from scratch. Then implement a basic inheritance chain three levels deep (Shape -> Polygon -> Rectangle) using only constructor functions and prototype assignment. Add a method at each level. Show that `instanceof` works correctly all the way up.

HARD

Reimplement `Array.prototype.map`, `filter`, and `reduce` from scratch and attach them to a custom `MyArray` constructor. Ensure they work identically to native versions including sparse array handling and the optional `thisArg` parameter. Write tests comparing `MyArray` results to native `Array` results.

■ The `this` Keyword in All Contexts

EASY

Create an object with a method that uses `this`. Call it four ways: as a method, as a detached function, with `.call()`, and as an arrow function assigned to the same key. Log `this` in each case and explain the result.

MEDIUM

Build a class `EventBus` with `on(event, handler)` and `emit(event, data)`. The handler must receive correct `this` when called as a callback inside `emit`. Show the bug that occurs when using regular functions as handlers in event listeners, then fix it correctly (not with `.bind` everywhere — use arrow functions in the right place).

HARD

Implement `Function.prototype.bind` from scratch as `myBind`. It must handle partial application, correct this binding even when the returned function is later called as a constructor with `new`, and pass the original function's `length` - bound args as the bound function's `length`.

■ Promises — Deep Understanding

EASY

Write three async operations (simulated with `setTimeout`) that must run in sequence: `fetchUser` -> `fetchUserPosts(userId)` -> `fetchPostComments(postId)`. Implement using only `.then()` chains (no `async/await`). Handle errors at each step.

MEDIUM

Implement `Promise.all`, `Promise.race`, `Promise.allSettled`, and `Promise.any` from scratch using only the `Promise` constructor (`new Promise`). Each must behave identically to the native version. Write failing test cases for each before implementing.

HARD

Build a `PromisePool(tasks, concurrency)` that runs an array of `Promise`-returning functions with a maximum of `N` running concurrently. When one completes, the next starts. Return all results in original order. Test with 20 tasks and concurrency of 3 — verify never more than 3 run at once.

■ async/await — Patterns and Pitfalls

EASY

Rewrite the `Promise` chain from the Promises easy assignment above using `async/await`. Then add a `try/catch` and demonstrate what happens if the middle request fails — which parts still execute?

MEDIUM

Write an `asyncRetry(fn, retries, delayMs)` wrapper that retries a failing async function up to `N` times with exponential backoff. On each retry, delay doubles. After all retries exhausted, throw the last error. Simulate a

flaky API that fails 70% of the time and show `asyncRetry` recovering.

HARD

Identify and fix all bugs in this scenario: a React-like component that fetches data in an `async` function, but the component may unmount before the fetch completes causing a `setState-on-unmounted-component` error. Implement a cancellable `async` operation using `AbortController` and show the cleanup. Then implement the same pattern using a manual `isCancelled` flag for environments without `AbortController`.

■ ES Modules — `import/export`, Circular Dependencies

EASY

Create a project with three modules: `mathUtils.js` (`add`, `multiply`), `stringUtils.js` (`capitalise`, `trim`), and `index.js` that imports and uses both. Use named exports everywhere. Then refactor to have a barrel file (`utils/index.js`) that re-exports everything. Confirm tree-shaking would work.

MEDIUM

Create a circular dependency between `moduleA` and `moduleB` (each imports from the other). Run it and document exactly what value you get and why. Then resolve the circular dependency by extracting shared logic to a third module. Explain the architectural lesson.

HARD

Build a plugin system where a core module dynamically imports plugins at runtime using `import()`. Each plugin exports a `register(core)` function. Core calls `register` on each loaded plugin. Show lazy loading: plugins only load when first needed, not at startup. Measure and log load time.

■ Generators and Iterators

EASY

Write a generator function `range(start, end, step)` that yields numbers like Python's `range()`. Use it in a `for...of` loop. Then consume it manually with `.next()` and log the `{ value, done }` objects.

MEDIUM

Implement an infinite Fibonacci generator. Then write a `take(n, iterable)` utility that pulls only the first `N` values from any iterable (works with your generator, arrays, `Sets`, `Maps`). Ensure it does not materialise the full sequence.

HARD

Build a custom iterable data structure: a `BinaryTree` class where `[Symbol.iterator]` does an in-order traversal using a generator. The tree must support `insert(value)`. Show that `[...tree]`, `for...of`, and `Array.from(tree)` all work correctly.

■ Destructuring, Spread, and Rest — Deep

EASY

Write a function `parseConfig({ host='localhost', port=3000, db: { name='mydb', pool=5 }={ } })` that destructures a nested config object with defaults. Call it with no args, partial args, and full args.

MEDIUM

Write a `mergeDeep(...objects)` function using spread that recursively merges nested objects (not just shallow). Arrays at the same key should be concatenated. Test with 3 levels of nesting.

HARD

Implement a `pick(obj, ...keys)` and `omit(obj, ...keys)` using destructuring and rest. Then write a `mapValues(obj, fn)` that applies `fn` to every value. None may use `lodash`. Write 10 test cases each covering edge cases: undefined values, symbol keys, inherited properties.

■ WeakMap, WeakRef, and Symbol

EASY

Use a `WeakMap` to attach private metadata to DOM-like objects (plain JS objects simulating nodes). Show that when the object is nulled, the `WeakMap` entry becomes eligible for GC (you cannot prove GC happened, but explain why it would).

MEDIUM

Build a private class fields system using `WeakMap` (pre-class-fields syntax). Implement a `Person` class where name and age are stored in a `WeakMap` keyed on the instance. No direct property access should expose them. Compare the memory semantics to actual private class fields (`#name`).

HARD

Create a plugin system that uses `Symbol.for()` for global registry keys and `Symbol()` for truly private keys. Build a middleware-like chain where each plugin identifies itself with a unique `Symbol`. Show that `Symbol.for('plugin:auth')` retrieves the same symbol across modules while local `Symbol()` does not.

■ Custom Error Types

EASY

Create three custom error classes: `ValidationError`, `NotFoundError`, and `AuthError`, all extending `Error`. Each must have a name, message, `statusCode`, and `stack`. Throw and catch each in a `try/catch` and verify `instanceof` works.

MEDIUM

Build a centralised error handler for an Express app. It must: differentiate your custom errors from unexpected errors, return appropriate HTTP status codes, not leak stack traces in production (`NODE_ENV=production`), and log the full error internally regardless.

HARD

Implement a `Result<T, E>` type in TypeScript (inspired by Rust) as a class that wraps either a success value or an error. It must have `.map()`, `.flatMap()`, `.unwrap()`, `.unwrapOr(default)`, and `.match({ok, err})`. Use it to eliminate all `try/catch` from a chain of three operations that each might fail.

■ Array.prototype Methods — Implement From Scratch

EASY

Implement `Array.prototype.map` and `Array.prototype.filter` from scratch. Attach them to a custom `MyArray` class. Show they produce identical output to native versions on the same input.

MEDIUM

Implement `Array.prototype.reduce`, `Array.prototype.flat`, and `Array.prototype.flatMap` from scratch. Handle edge cases: `reduce` with no initial value on empty array should throw, `flat` should handle depth parameter including `Infinity`.

HARD

Implement `Array.prototype.sort` using merge sort underneath. It must be stable (equal elements preserve original order), accept a custom comparator, and handle arrays with undefined elements identically to V8's native sort. Prove stability with a test case.

— TypeScript —

■ Generics with Constraints

EASY

Write a generic function `getProperty<T, K extends keyof T>(obj: T, key: K): T[K]`. Call it with a `User` object. TypeScript must infer the return type automatically.

MEDIUM

Build a generic `Repository<T extends { id: string }>` class with `findById(id)`, `findAll()`, `save(entity: T)`, and `delete(id)`. Implement it twice: once with an in-memory `Map`, once with a JSON file. Both use the same interface.

HARD

Implement a type-safe `EventEmitter<Events extends Record<string, unknown[]>>` where `Events` is a map of event name to argument tuple types. `on()` and `emit()` must be fully typed — calling `emit('login', user)` where `login` is typed as `[User]` should fail if you pass a string instead.

■ Discriminated Unions

EASY

Model an API response as a discriminated union: `type Result<T> = { status: 'ok', data: T } | { status: 'error', code: number, message: string }`. Write a `handleResult` function that narrows correctly with a switch on status.

MEDIUM

Model a state machine for an order: `Pending -> Processing -> Shipped -> Delivered | Cancelled`. Each state is a discriminated union member with state-specific fields (e.g. `Shipped` has `trackingNumber`). Write transition functions that TypeScript enforces at compile time — you cannot transition to an invalid state.

HARD

Build a type-safe command pattern where commands are a discriminated union and a handler registry maps each command type to a handler typed to that exact command's payload. Adding a new command variant must cause a TypeScript error if no handler is registered (use exhaustive checks).

■ Utility Types — Deep Usage

EASY

Given a `User` type with 10 fields, create: `UserPreview` (only name and avatar), `UserUpdate` (all fields optional), `UserCreateInput` (everything except id and createdAt), `ReadOnlyUser`. Use only built-in utility types.

MEDIUM

Write your own implementations of `Partial<T>`, `Required<T>`, `ReadOnly<T>`, `Pick<T, K>`, `Omit<T, K>`, and `Record<K, V>` without using the built-in versions. Name them `MyPartial` etc. Verify they behave identically.

HARD

Implement `DeepPartial<T>`, `DeepReadOnly<T>`, and `DeepRequired<T>` that work recursively through nested objects and arrays. Test with a type 5 levels deep. Then implement `NonNullableDeep<T>` that removes null and undefined at every level.

■ The `infer` Keyword

EASY

Use `infer` to write `UnpackPromise<T>` that extracts the resolved type from a `Promise<T>`. Test: `UnpackPromise<Promise<User>>` should equal `User`.

MEDIUM

Write `FirstArg<T>` and `SecondArg<T>` that extract the first and second parameter types from a function type. Write `ReturnType<T>` yourself without using the built-in. Then write `Promisify<T>` that wraps every function in an object to return a `Promise`.

HARD

Implement a `UnionToIntersection<U>` type that converts a union `A | B | C` into `A & B & C` using `infer` in a contravariant position. Explain why the contravariant trick works.

■ Declaration Files (.d.ts)

EASY

Write a declaration file for a hypothetical JS library called `stringHelpers` that exports: `truncate(str, len): string`, `slugify(str): string`, and a `Formatter` class with a `format(template, data)` method.

MEDIUM

Take a real untyped npm package (e.g., a small utility without `@types`). Write a complete `.d.ts` declaration file for it by reading the source. Augment it with module augmentation to add a custom method that you monkey-patched onto it in your project.

HARD

Build a typed plugin system using declaration merging. Define a core module with an empty `PluginRegistry` interface. Each plugin augments `PluginRegistry` to declare its own namespace. The host reads `PluginRegistry` and all registered plugins are fully typed without the host knowing about them at authorship time.

■ The `satisfies` Operator

EASY

Define a `palette` object and use `satisfies Record<string, string>` to get both type safety and literal type inference on the values (so `palette.primary` stays `'blue'` not just `string`).

MEDIUM

Use `satisfies` to validate a route config object against a `RouteConfig` type while preserving the exact shape. Show how this differs from a type annotation (which widens) and a cast (which is unsafe).

HARD

Build a configuration system where a large config object satisfies a `Config` schema at compile time but retains its literal types for downstream use. Show three places where `satisfies` is strictly better than `: Config` annotation, with tests that fail at compile time if config is invalid.

■ TypeScript Strict Mode — All Flags

EASY

Enable `strictNullChecks` on an existing JS-style codebase (write one with 5 functions that assume values are never null). Fix every error. Document what each fix prevented at runtime.

MEDIUM

Enable `noUncheckedIndexedAccess`. Update a codebase that reads from arrays and object indexes without checking for undefined. Fix every access safely using optional chaining or explicit checks. Explain why this flag matters in production.

HARD

Create a `tsconfig.json` with every strict-family flag enabled: `strict`, `noUncheckedIndexedAccess`, `exactOptionalPropertyTypes`, `noImplicitOverride`, `useUnknownInCatchVariables`. Take a 200-line loosely typed file and bring it to full compliance. Document every change and its runtime safety implication.

LAYER 2

— Node.js Internals —

■ Event Loop Phases in Node.js

EASY

Write a Node script that uses `setImmediate`, `setTimeout(fn,0)`, `process.nextTick`, and a resolved `Promise.then()`. Log each as it fires. Before running, write the exact expected order. Explain which phase each fires in.

MEDIUM

Build a benchmarking tool that measures the overhead of different async scheduling primitives across 100,000 iterations each: `setTimeout(fn,0)`, `setImmediate`, `process.nextTick`, and `Promise.resolve().then()`. Show the latency histogram for each.

HARD

Write a Node HTTP server that processes requests in a way that starves the I/O phase — demonstrate the starvation by placing heavy computation in a tight loop. Then fix it using `setImmediate` to yield to the event loop, and show the request latency difference in a before/after benchmark.

■ Node.js Streams

EASY

Read a large text file (generate a 50MB file of random lines) using a `Readable` stream. Count the number of lines without loading the file into memory. Log memory usage before and after.

MEDIUM

Build a file processing pipeline: `ReadableStream` -> `TransformStream` (converts lines to uppercase) -> `TransformStream` (filters lines containing a keyword) -> `WritableStream` (writes to output file). Use `pipe()` and handle backpressure. Test with a 100MB file.

HARD

Implement a custom `Transform` stream that parses a CSV byte-by-byte, handles quoted fields with commas inside, emits parsed row objects, and handles malformed rows without crashing. Pipe it through `gzip` compression to a file and verify the output can be decompressed and re-parsed.

■ Buffers

EASY

Read a binary file (any image) into a `Buffer`. Log its length in bytes, extract the first 10 bytes as hex, and write it back out to a new file. Verify the new file is identical using a checksum.

MEDIUM

Implement a simple binary protocol: encode a message as [1-byte type | 4-byte length | N-byte payload] into a `Buffer`. Write both `encode(type, payload)` and `decode(buffer)` functions. Round-trip 10 messages and verify perfect reconstruction.

HARD

Build a Buffer pool that pre-allocates a fixed amount of memory and hands out slices on request. When a slice is returned, it must be zeroed and made available again. This should never allocate new memory after initialisation. Show it handling 1000 concurrent allocations without exceeding the pool size.

■ **child_process — exec, spawn, fork**

EASY

Use `child_process.exec` to run `git log --oneline -10` in the current directory and print the result. Handle the case where git is not installed gracefully.

MEDIUM

Use `spawn` to run a long-running child process (a script that prints a line every second for 30 seconds). Stream its stdout to the parent's stdout in real time. Kill the child after 5 lines using `child.kill()`. Show that `SIGTERM` is received by the child.

HARD

Build a worker farm: a parent process that spawns `N` child processes using `fork()`. Distribute an array of CPU-heavy tasks (computing `isPrime` for large numbers) across children via IPC. Collect results as children finish. Show that `N` children saturate `N` CPU cores and completes faster than single-process execution. Measure the speedup.

■ **cluster Module**

EASY

Write an HTTP server. Run it with `cluster` so it forks one worker per CPU core. Log which PID handles each request. Use `ab` or `wrk` to send 1000 requests and show they are distributed across workers.

MEDIUM

Add graceful restart to your cluster setup: when the master receives `SIGUSR2`, it restarts workers one at a time (rolling restart), ensuring the server never has zero workers running. Verify with continuous load that no requests fail during a restart.

HARD

Build a cluster-aware session store: sessions created in one worker must be readable in another. Implement this using Redis as the shared store. Show that sticky sessions are not needed because state lives in Redis, not in process memory.

■ **EventEmitter**

EASY

Build a simple logger that extends `EventEmitter`. Emit `'log'`, `'warn'`, and `'error'` events. Register listeners for each that write to different outputs (console, a file). Demonstrate that multiple listeners on the same event all fire.

MEDIUM

Build a typed event bus class in TypeScript where event names and their payload types are declared in a generic map. `on()` and `emit()` must be fully type-safe — passing wrong payload type is a compile error. Implement `once()` that auto-removes after one fire.

HARD

Investigate and fix an EventEmitter memory leak: build a scenario where listeners are added inside a loop and never removed, triggering the MaxListenersExceededWarning. Fix it three ways: removeListener, once, and using AbortSignal-based auto-cleanup. Show memory usage stabilising after the fix using process.memoryUsage().

■ The process Object

EASY

Read command-line arguments from process.argv and environment variables from process.env. Build a small CLI that accepts --port and --env flags with fallback to env vars, then to defaults. Print its resolved config on start.

MEDIUM

Handle process signals gracefully: on SIGTERM and SIGINT, your HTTP server must stop accepting new connections, wait for in-flight requests to complete (up to 10 seconds), close the DB connection pool, then exit with code 0. Simulate with a slow endpoint that takes 3 seconds.

HARD

Build a process health monitor: every 5 seconds log memory usage (heapUsed, heapTotal, rss, external), CPU usage (using process.cpuUsage()), event loop lag (using a setImmediate timer trick), and active handles count. Format as structured JSON. Show the metrics spiking during a CPU-bound operation.

— Express.js —

■ Router Organisation — Feature-Based Architecture

EASY

Refactor a single-file Express app with 10 routes into a feature-based structure: each feature (users, posts, comments) has its own router file. Mount them in app.js. Verify all routes still work.

MEDIUM

Build a route registration system that auto-discovers router files in a /routes directory and mounts them at paths derived from their filenames (routes/users.js -> /api/users). Add support for versioning: routes/v1/users.js -> /api/v1/users.

HARD

Design and implement a full layered architecture for a blog API: route handlers only call service methods, services contain business logic and call repositories, repositories handle all DB queries. No DB query may appear outside a repository file. Enforce this with a lint rule or test that fails if an import violation occurs.

■ Error Handling Middleware

EASY

Add a 4-argument error handler to Express. Throw errors from 3 different routes. Verify the handler catches all of them and returns { error: message } as JSON.

MEDIUM

Build a centralised error handler that handles: ValidationError (400), NotFoundError (404), AuthError (401), and all unknown errors (500). In development, include the stack trace in the response. In production, log the full error but return only a generic message.

HARD

Implement async error propagation without try/catch in every route: write an `asyncHandler(fn)` wrapper that catches Promise rejections and passes them to `next()`. Apply it to all routes. Then implement a global `unhandledRejection` listener as a final safety net. Write a test that proves an async error in a route still reaches the error handler.

■ Request Validation with Zod

EASY

Write a Zod schema for a POST `/users` body: name (min 2 chars), email (valid email), age (number, 18-120). Validate it in middleware. Return structured errors if validation fails.

MEDIUM

Build a generic `validate(schema)` middleware factory that validates request body, query params, and URL params separately using three different Zod schemas passed as arguments. Return all validation errors at once (not just the first one). Type the middleware so TypeScript knows the validated shapes downstream.

HARD

Implement a request coercion layer: query params arrive as strings, but your Zod schema expects numbers and booleans. Use Zod's `coerce` to handle conversion. Write 20 test cases covering edge cases: `'0' -> false` or `0`, `'true' -> true`, `'NaN' -> rejection`, empty string vs missing field, arrays from repeated query keys (`?tag=a&tag=b`).

■ File Uploads with Multer + S3

EASY

Accept a single image upload with multer using memory storage. Validate that it is an image (MIME type check). Return the file size and original name in the response.

MEDIUM

Build a file upload endpoint that: validates file type (images and PDFs only), limits size to 5MB, renames to a UUID to prevent collisions, and stores to a local `/uploads` directory. On success, return a URL to retrieve the file. Serve the `/uploads` directory as static files.

HARD

Replace local storage with S3 (use MinIO locally — it is S3-compatible and free). Generate a presigned upload URL on the backend that the client uses to upload directly to S3 (client never sends the file through your server). After upload, client notifies your server, you verify the object exists in S3, and save the URL to the database.

■ Consistent API Response Structure

EASY

Define a standard API response shape: `{ success, data, error, meta }`. Write helper functions `sendSuccess(res, data, statusCode)` and `sendError(res, message, statusCode)`. Use them in 5 routes.

MEDIUM

Build a response interceptor that automatically wraps every Express response in your standard shape by monkey-patching `res.json`. Existing routes that call `res.json(data)` should transparently get wrapped without modification.

HARD

Design a hypermedia API (HATEOAS-lite): each response includes a `_links` object with relevant actions the client can take next. For a `/users/:id` response, `_links` includes `self`, `update`, `delete`, and `posts`. For a `/users` list, each item has its own `_links`. Implement this generically via a link-builder that takes the resource type and ID.

— API Design —

■ REST Conventions and Status Codes

EASY

Build a `/articles` CRUD API that correctly uses: 201 for create, 200 for read/update, 204 for delete, 400 for validation failure, 404 for not found, 409 for conflict (duplicate slug). Write a test for each status code.

MEDIUM

Design the URL structure for a nested resource API: users have posts, posts have comments, comments can be liked. Implement all CRUD endpoints following REST conventions. Discuss where you break nesting (e.g. `/likes` rather than `/users/:id/posts/:id/comments/:id/likes`) and why.

HARD

Implement a fully RESTful API with HATEOAS, ETag-based caching (return 304 Not Modified when ETag matches), and conditional updates (If-Match header to prevent lost updates). Write a test that demonstrates the optimistic locking: two concurrent updates to the same resource — second one should get 412 Precondition Failed.

■ Pagination — Offset vs Cursor

EASY

Implement offset-based pagination on a `/posts` endpoint: accept `page` and `limit` query params, return `{ data, total, page, totalPages }` in the response.

MEDIUM

Implement cursor-based pagination on the same endpoint: accept `cursor` (an encoded last-seen ID) and `limit`. Return `{ data, nextCursor }` where `nextCursor` is null on the last page. Show why this is more stable than offset when records are inserted during pagination.

HARD

Implement keyset pagination with sorting support: the client can sort by `createdAt` or `title`, and the cursor encodes both the sort value and the ID to handle ties. The query must use an index and not do a full table scan even on page 1000. Prove with EXPLAIN ANALYZE that the query uses an index at any page.

■ API Versioning

EASY

Add v1 and v2 to your API where v2 of GET `/users` returns camelCase fields while v1 returns snake_case. Support versioning via URL path (`/api/v1/`, `/api/v2/`).

MEDIUM

Support three versioning strategies simultaneously: URL path, Accept header (Accept: `application/vnd.api.v2+json`), and custom header (`X-API-Version: 2`). The URL takes priority, then header, then Accept. Write a version-resolution middleware.

HARD

Implement a version deprecation system: v1 endpoints return a Deprecation header with sunset date, log warnings when deprecated endpoints are called, and after the sunset date return 410 Gone. Track version usage per endpoint in Redis so you know which old versions are still in use before removing them.

■ OpenAPI / Swagger Documentation

EASY

Write an OpenAPI 3.0 spec by hand for your /users CRUD API. Include all request bodies, response schemas, and error responses. Serve it via swagger-ui-express.

MEDIUM

Use zod-to-openapi or similar to auto-generate OpenAPI docs from your existing Zod schemas. Every route must have documented request/response schemas that stay in sync automatically when you change the Zod schema.

HARD

Set up contract testing: generate a mock server from your OpenAPI spec and run your frontend against the mock. Then run your backend against the spec with dredd or similar. Any drift between spec and implementation fails the CI pipeline.

— Authentication and Authorization —

■ Sessions vs JWT — Tradeoffs

EASY

Build the same login system twice: once with server-side sessions (express-session + Redis store) and once with JWTs. Demonstrate that both protect a /me route. List three tradeoffs of each approach.

MEDIUM

Show JWT revocation problem: issue a JWT, let the user log out, then show the token still works until expiry. Fix this with a Redis token blacklist. Then show why sessions don't have this problem. Document the operational cost of each approach at 1M users.

HARD

Design and implement a hybrid approach: short-lived JWTs (15 min) for stateless auth plus refresh token rotation stored in Redis. On each refresh, issue new access + refresh tokens and invalidate the old refresh token. Implement refresh token families — if a stolen refresh token is used, invalidate the entire family (detect theft).

■ OAuth2 with PKCE — Login with Google

EASY

Implement 'Login with Google' using the OAuth2 authorization code flow. Store the returned access token and create a local user session. Display the user's name and avatar.

MEDIUM

Add GitHub OAuth alongside Google. Abstract the OAuth logic so adding a third provider requires only a config object (clientId, clientSecret, authUrl, tokenUrl, userInfoUrl). Normalise the user profile from each provider into a consistent shape.

HARD

Implement the full OAuth2 PKCE flow from scratch without using passport.js: generate code_verifier and code_challenge, include in the authorization URL, send code_verifier on token exchange, verify server-side. Explain what PKCE prevents and demonstrate the attack it stops (intercept the auth code).

■ Role-Based Access Control (RBAC)

EASY

Add roles (admin, editor, viewer) to your user model. Write an authorize(role) middleware that rejects requests from users without the required role. Protect 3 routes.

MEDIUM

Implement permission-based RBAC: roles have sets of permissions (e.g. admin has user:delete, post:publish; editor has post:create, post:edit; viewer has post:read). Middleware checks for specific permissions, not roles. Adding a new permission to a role in the database takes effect immediately without redeployment.

HARD

Build a hierarchical RBAC system: super-admin > admin > manager > user. Higher roles inherit all permissions of lower roles. Permissions can also be granted or revoked per-user overriding their role. Resource-level permissions: a user can edit their own posts but not others'. Implement this without hardcoding any role-to-permission mapping in code.

■ Password Security — Hashing and Reset Flow

EASY

Hash passwords with bcrypt (cost factor 12) on registration. Verify on login. Show that two registrations with the same password produce different hashes.

MEDIUM

Implement a password reset flow: user requests reset via email, a single-use token (stored hashed in DB with 1-hour expiry) is emailed, user sets new password with token, token is consumed and all existing sessions are invalidated.

HARD

Implement the Have I Been Pwned k-anonymity check: on registration, send the first 5 characters of the password's SHA1 hash to the HIBP API, check if the full hash suffix appears in the response, and reject the password if it has been breached. Do this without sending the full password or hash to any external service.

■ Email Verification and 2FA / TOTP

EASY

After registration, send a verification email with a signed token link. Mark the user as verified when the link is clicked. Block unverified users from sensitive endpoints.

MEDIUM

Implement TOTP-based 2FA: on enrollment, generate a secret, show a QR code the user scans with Google Authenticator, verify the first code to confirm setup. On login, require the TOTP code as a second factor. Allow recovery codes (one-time use, stored hashed).

HARD

Implement full 2FA with: TOTP support, backup codes (8 single-use codes, hashed, regeneratable), trusted device cookies (skip 2FA for 30 days on the same device), and admin ability to forcibly disable 2FA for account recovery. Security log every 2FA event with timestamp and IP.

— Background Jobs, WebSockets —

■ BullMQ — Job Queues

EASY

Set up BullMQ with Redis. Create a queue with an email-sending worker. Add a job from an API endpoint. Show the job is processed by the worker. Log job start, completion, and failure.

MEDIUM

Build a job pipeline: when a user uploads a video, add a job to a transcoding queue. The job runs ffmpeg (or simulates it with a sleep), updates the database with progress, then adds a thumbnail-generation job on completion. Implement retry with exponential backoff and a dead-letter queue for jobs that fail all retries.

HARD

Build a distributed job processing system: multiple worker processes (use cluster or separate Node processes) consume from the same BullMQ queue. Implement job prioritisation (premium user jobs get higher priority), rate limiting per user (max 5 jobs/min), and job deduplication (if an identical job is already queued, do not add another). Monitor queue health with BullMQ's `getMetrics()`.

■ WebSockets with Socket.io

EASY

Build a real-time chat room: users join a named room, messages broadcast to all users in the room, show when users join and leave. Test with two browser tabs.

MEDIUM

Add authentication to your WebSocket server: the client sends a JWT on connection, the server validates it and attaches the user to the socket. Rooms are private — a user can only join rooms they have permission to access (check against DB). Implement a typing indicator that emits 'user is typing' with debounce.

HARD

Build a collaborative document editing system: multiple users edit a shared text document. Implement operational transformation (simplified: last-writer-wins per line) to handle concurrent edits. Show cursor positions of other users in real time. Handle reconnection gracefully — client syncs to current document state on reconnect without losing its in-flight changes.

■ Server-Sent Events (SSE) vs WebSockets vs Polling

EASY

Implement a live dashboard using SSE: the server pushes a new data point every second to all connected clients. The client renders it in real time. Compare to a polling approach and measure the difference in HTTP requests per minute.

MEDIUM

Build a notification system using SSE: users connect and receive real-time notifications. When a notification is sent via a REST endpoint, it is pushed to all connected clients for that user (user may have multiple tabs open). Handle reconnection with Last-Event-ID so missed events are replayed.

HARD

Implement the same real-time feature three ways: long-polling, SSE, and WebSockets. Benchmark all three under 500 concurrent connections measuring: memory per connection, latency from server event to client receipt, CPU on server during burst. Write a decision guide explaining which to use for: notifications, chat, live collaborative editing, real-time dashboards.

LAYER 3

Most full-stack devs are weak here. This layer will separate you.

■ SQL Fundamentals — JOINS, CTEs, Subqueries

EASY

Given tables: users, posts, comments. Write queries to: get all posts with their author name, get all users who have no posts, get the 5 most commented posts.

MEDIUM

Write a query using a CTE that: first calculates each user's post count, then ranks users by post count per month, then returns only users who were in the top 10 for at least 3 months. Use window functions (RANK, ROW_NUMBER, LAG).

HARD

Implement a recursive CTE for a hierarchical category tree (each category has a parent_id). Write a query that returns the full path from root to leaf for any category (e.g. Electronics > Phones > Smartphones). Write a second query that returns all descendants of a given category.

■ Database Design and Normalization

EASY

Design a schema for an e-commerce system: products, categories (many-to-many), customers, orders, order items. Write the CREATE TABLE statements with appropriate constraints and foreign keys.

MEDIUM

Start with a denormalized table: orders with columns customer_name, customer_email, product_name, product_sku, product_price, quantity, total. Normalize it to 3NF. Show the before/after schema and write the migration.

HARD

Design a multi-tenant SaaS database: all tenants share tables but data must be strictly isolated. Evaluate three approaches: shared tables with tenant_id column, row-level security (PostgreSQL RLS), and separate schemas per tenant. Implement RLS. Show that a query in tenant A's session cannot return tenant B's data even if the SQL doesn't filter by tenant_id.

■ Indexes — B-tree, GIN, Partial, Composite, Covering

EASY

Add indexes to a posts table with 100,000 rows. Use EXPLAIN ANALYZE to show before/after for: search by user_id, search by created_at range, and search by status.

MEDIUM

Identify the correct index for each scenario: (a) WHERE status = 'active' AND created_at > '2024-01-01' — composite or two separate? (b) WHERE email ILIKE '%@gmail.com' — what type of index helps? (c) JSONB column with arbitrary keys — GIN index. Implement all three and prove with EXPLAIN ANALYZE that each is used.

HARD

Build a full-text search feature on a posts table (title + body). Use tsvector with a GIN index. Support: multi-word search, phrase search, ranking by relevance (ts_rank), and highlighting matched terms (ts_headline). Benchmark against ILIKE search on the same data — show the order-of-magnitude difference.

■ EXPLAIN ANALYZE — Reading Query Plans

EASY

Write an intentionally slow query (no indexes, joining 3 tables, sorting unsorted data). Run EXPLAIN ANALYZE. Identify the Seq Scan, Sort, and Hash Join nodes. Add the right index and run again — show the Seq Scan becoming an Index Scan.

MEDIUM

Given a slow query in production (simulate one: a report query joining 5 tables with GROUP BY and ORDER BY on unindexed columns), read the EXPLAIN ANALYZE output and identify the three most expensive nodes. Add indexes and rewrite using a CTE to fix each one. Show the execution time improvement.

HARD

Benchmark the same query under three conditions: cold cache (pg_prewarm off), warm cache, and with connection pooling (PgBouncer in transaction mode). Show how the plan changes (Bitmap Heap Scan vs Index Only Scan) with different table statistics. Run ANALYZE and show the plan improving.

■ Transactions, ACID, and Isolation Levels

EASY

Write a bank transfer in a transaction: deduct from account A, add to account B. Show that if the credit fails, the debit is rolled back. Verify with a test.

MEDIUM

Demonstrate each isolation level anomaly: (a) dirty read — use two concurrent transactions in READ UNCOMMITTED (simulate if PG doesn't support), (b) non-repeatable read in READ COMMITTED, (c) phantom read in REPEATABLE READ. Show that SERIALIZABLE prevents all three. Use two concurrent psql sessions.

HARD

Implement optimistic locking with a version column: on update, increment version and require the incoming version to match. If another transaction updated first, return a conflict error. Build a test that fires 100 concurrent updates to the same row and shows exactly one succeeds per round, all others get conflict errors, and no update is silently lost.

■ Database Migrations

EASY

Write forward and backward migrations for: adding a column, renaming a column, adding an index, and adding a foreign key. Run forward then backward and verify the schema matches.

MEDIUM

Write a zero-downtime migration for adding a NOT NULL column to a large table: (1) add column as nullable, (2) backfill existing rows in batches, (3) add NOT NULL constraint. Show why doing it in one step locks the table and how the three-step approach avoids a lock.

HARD

Implement a migration that changes a one-to-many relationship to many-to-many (e.g. posts have one category -> posts have many categories). Write the migration so it is data-safe: existing data is preserved, the old column is only dropped after the new table is populated. Run it against a database with 10,000 rows and verify no data loss.

■ Prisma ORM — Schema, Relations, N+1

EASY

Define a Prisma schema for a blog: User, Post, Comment, Tag. Include all relations. Run `prisma migrate dev` and `prisma db seed` with 100 fake records.

MEDIUM

Write Prisma queries for: get a post with its author and all comments with their authors in one query. Then write the same without using `include` — show the N+1 queries in the logs. Fix the N+1 and verify the query count drops to 1.

HARD

Build a full repository layer using Prisma where raw SQL is used for one complex reporting query (using `prisma.$queryRaw` with tagged template literals for SQL injection safety). Implement cursor-based pagination using Prisma's `cursor` argument. Set up Prisma query logging and prove the N+1 is gone by counting logged queries.

■ Redis — Data Structures and Caching

EASY

Cache the result of a slow database query in Redis with a 60-second TTL. On the second request, return from cache. Log which path was taken.

MEDIUM

Implement a leaderboard using Redis Sorted Sets (`ZADD`, `ZREVRANGE`, `ZRANK`). Support: add/update a score, get top 10, get a user's rank, get users near a given user's rank. All operations must be $O(\log N)$.

HARD

Build a distributed rate limiter using Redis: implement sliding window rate limiting (not fixed window) using a Sorted Set per user where members are request timestamps. On each request, remove expired timestamps, count remaining, reject if over limit. Wrap in a Lua script to make the check-and-increment atomic. Test with 100 concurrent requests from the same user.

■ Redis — Pub/Sub and Session Store

EASY

Use Redis Pub/Sub to send a message from one Node process to another. Publisher sends, subscriber receives and logs. Show they are separate processes.

MEDIUM

Implement a real-time notification fan-out: when a new post is published, publish to a Redis channel. All connected Socket.io server instances (simulated with two Node processes) receive the event and push it to their connected clients. This solves the multi-server Socket.io problem.

HARD

Replace Redis Pub/Sub with Redis Streams for the notification system. Streams provide: message persistence (Pub/Sub does not), consumer groups (each message processed by exactly one consumer in a group), and `XPENDING` to detect and retry unacknowledged messages. Implement a consumer group with 3 workers and show that each message is processed exactly once.

LAYER 4

■ All React Hooks — Deep Understanding

EASY

Build a stopwatch using `useState` and `useEffect`. Start, stop, reset, and lap. Show that cleanup in `useEffect` is essential — remove it and demonstrate the memory leak.

MEDIUM

Build a `useDebounce(value, delay)` custom hook. Use it to debounce a search input that calls an API. Show the API is only called 300ms after the user stops typing, not on every keystroke. Compare the request count with and without debouncing.

HARD

Implement `useReducer` for a shopping cart with complex state transitions: `ADD_ITEM` (increase quantity if exists), `REMOVE_ITEM`, `UPDATE_QUANTITY`, `APPLY_COUPON`, `CLEAR_CART`. Use `useContext` to provide the cart globally. Write a custom `useCart()` hook. Prove that cart state persists across re-renders and that `useReducer` is correct when multiple actions fire in rapid succession.

■ Custom Hooks — Extracting and Composing Logic

EASY

Extract a `useLocalStorage(key, initialValue)` hook. It reads from `localStorage` on mount and writes on change. Works identically to `useState` from the consumer's perspective.

MEDIUM

Build `useAsync(asyncFn, deps)`: a hook that manages the loading/error/data states for any async operation. It re-runs when `deps` change and cancels the previous call if `deps` change before completion. Returns `{ data, loading, error, refetch }`.

HARD

Build a `useWebSocket(url)` hook that manages the connection lifecycle: connect on mount, reconnect with exponential backoff on disconnect, expose `send(message)` and the last received message. Handle the case where `send()` is called before the connection is open (queue messages). Expose connection status (connecting/open/closed). Clean up on unmount.

■ React Performance — `memo`, `useMemo`, `useCallback`

EASY

Create a parent that re-renders every second. It has a child that does an expensive render (simulate with a slow loop). Wrap the child in `React.memo`. Verify in React DevTools Profiler that the child stops re-rendering unnecessarily.

MEDIUM

Build a data table with 1000 rows. Clicking a row highlights it. Without optimisation, all 1000 rows re-render on each click. Use `React.memo` + `useCallback` on the row click handler to ensure only 2 rows re-render (the

newly highlighted and previously highlighted). Prove with the Profiler.

HARD

Profile a real component tree with the React DevTools Profiler. Identify the 3 slowest components. Apply the minimum necessary optimisations (do not over-use memo). Document each optimisation decision: why this component, what triggered unnecessary renders, why the fix works, what the before/after render time is.

■ Error Boundaries, Suspense, and Lazy Loading

EASY

Create an `ErrorBoundary` class component. Wrap a component that throws. Show the error boundary renders a fallback UI instead of crashing the whole app.

MEDIUM

Lazy-load 5 page components using `React.lazy` and `Suspense`. Show in the Network tab that each page's JS chunk only loads when first visited. Add a meaningful loading skeleton as the `Suspense` fallback.

HARD

Build nested `Suspense` boundaries where each section of the page loads independently: sidebar loads, main content loads, recommended section loads — each with its own skeleton. Combine with an `ErrorBoundary` per section so one section failing does not kill the whole page. Simulate network delays to show the cascading skeleton-to-content transitions.

■ Zustand — State Management

EASY

Manage a global theme (light/dark) with `Zustand`. Toggle from any component. Persist the preference to `localStorage` using the `persist` middleware.

MEDIUM

Build a notification system with `Zustand`: add, dismiss, and auto-expire notifications. Multiple components can add notifications. A single `NotificationContainer` reads from the store and renders them. Show that adding a notification from a non-React context (e.g. an axios interceptor) works correctly.

HARD

Build a `Zustand` store for a multi-step form wizard with 5 steps. Each step has its own validation. State includes: current step, all form data, validation errors per step, and submission status. Going back preserves data. Implement undo/redo using `Zustand`'s temporal middleware (`zundo`). Persist the entire wizard state to `sessionStorage` so a page refresh returns to the same step.

■ TanStack Query — Server State Management

EASY

Replace a `useEffect` fetch with `useQuery`. Show automatic loading/error states, background refetching when the window regains focus, and that navigating away and back does not cause an unnecessary network request (data is cached).

MEDIUM

Build a paginated list with `usePaginatedQuery` (or `useInfiniteQuery` for infinite scroll). Implement optimistic updates for a delete action: remove the item from the UI immediately, then rollback if the API call fails. Show the rollback working by simulating a failed request.

HARD

Implement a complex data dependency: first fetch a project, then in parallel fetch the project's members and tasks (dependent queries using `enabled: !!projectId`). Use `queryClient.prefetchQuery` to preload the project detail when hovering a list item. Implement a mutation that invalidates multiple related queries on success. Show the cache state in React Query DevTools throughout.

■ **React Hook Form + Zod**

EASY

Build a registration form with name, email, and password. Validate with Zod (email format, password min 8 chars). Show real-time error messages. Prevent submission if invalid.

MEDIUM

Build a multi-step form with 3 steps. Each step has its own Zod schema. Validation only runs for the current step. Going back preserves values. On final submit, show all data merged and submitted in one API call.

HARD

Build a dynamic form builder: the form fields are defined by a JSON config fetched from the API. Each field has a type (text, select, checkbox, date, file), label, name, and validation rules also expressed in JSON. The Zod schema is built dynamically from the config. The form re-renders when the config changes without losing existing values.

■ **Tailwind CSS + shadcn/ui**

EASY

Rebuild a design you like (a card, a navbar, a hero section) using only Tailwind utility classes. Make it fully responsive: looks good on mobile (375px), tablet (768px), and desktop (1280px).

MEDIUM

Install shadcn/ui. Build a data table with: sorting by column, filtering by a text input, pagination, row selection, and a bulk-delete action. Use shadcn's Table, Input, Button, and Checkbox components. Make the table state (sort, filter, page) persist in URL query params.

HARD

Build a full dashboard layout with: collapsible sidebar, topbar with user menu and notifications, a data grid with charts (use Recharts), and a command palette (Cmd+K) for quick navigation. Implement dark mode with shadcn's ThemeProvider. Every component must work in both modes without a single hardcoded color.

■ **React Router v6 — Nested Routes, Loaders, Actions**

EASY

Set up React Router v6 with 5 routes: Home, Login, Dashboard, Profile, and a 404. Protect the Dashboard and Profile routes with an auth check that redirects to Login.

MEDIUM

Implement nested routes for a dashboard: `/dashboard` shows a layout with sidebar, `/dashboard/users` shows users, `/dashboard/posts` shows posts — both rendered inside the same layout. Add a default redirect from `/dashboard` to `/dashboard/users`.

HARD

Use React Router v6 Data APIs: implement a loader that fetches a user before the Profile page renders (no loading spinner — data is ready on mount). Implement an action that handles a form submission using Form and redirect on success. Handle loader errors with an ErrorBoundary that shows the loader's error message.

LAYER 5

■ Server Components vs Client Components

EASY

Build a page with a Server Component that fetches data directly from the database (no API call) and a Client Component for an interactive like button. Show in the Network tab that the server component produces no client JS bundle.

MEDIUM

Build a product listing page entirely as a Server Component (async data fetching, no `useEffect`). Add a client-side filter UI that, when used, triggers a new server render via URL search params (not client-side filtering). Verify filtering works without any client-side state holding the product data.

HARD

Identify the component boundaries in a complex page: a comments section needs real-time updates (client), a sidebar with stats can be server-rendered, a post body is server-rendered but has a client-side copy button. Implement all three correctly. Profile the resulting bundle: the client JS should only include the two interactive islands.

■ Server Actions — Forms and Mutations

EASY

Build a contact form using a Server Action. On submit, save to the database. Show that no API route is needed. Add `useFormState` to display server-side validation errors inline.

MEDIUM

Build an optimistic UI with a Server Action: clicking 'Like' immediately shows the incremented count (`useOptimistic`), fires the Server Action, and rolls back if it fails. Add `revalidatePath` so the cached page is invalidated after a successful action.

HARD

Build a file upload with a Server Action: the form submits `multipart/form-data`, the action receives the `File`, uploads it to S3, and saves the URL to the database. Show progress by streaming partial responses. Add input validation using `Zod` inside the Server Action and return typed errors. Handle concurrent uploads: multiple files, each tracked independently.

■ Rendering Strategies — SSR, SSG, ISR, CSR

EASY

Create four routes demonstrating each rendering strategy: SSG (blog post, static at build time), SSR (user dashboard, per request), ISR (product page, regenerate every 60s), CSR (interactive chart, client only). Inspect each in the Network tab and identify which sends HTML vs empty shell.

MEDIUM

Build a product catalogue: use SSG for product detail pages (pre-generated at build), ISR for the product listing (prices update every minute), and SSR for the checkout page (real-time stock check per request). Explain why each choice is correct.

HARD

Implement selective hydration: a marketing page is fully static (SSG) except for a personalised greeting that shows the logged-in user's name. Solve this without making the whole page SSR: use a client component island just for the greeting, with a loading skeleton that shows during hydration. The page's Lighthouse score must remain in the 90s.

■ Next.js Middleware

EASY

Write middleware that redirects all unauthenticated requests to /login, except for the /login and /api/auth routes. Test that protected pages redirect correctly.

MEDIUM

Build A/B testing with middleware: on first visit, randomly assign the user to variant A or B (store in a cookie). The middleware rewrites the URL to /home-a or /home-b transparently. Log which variant is shown. Show that the user stays in the same variant on refresh.

HARD

Build a feature flag system using middleware: flags are stored in an Edge Config (or a fast KV store). Middleware checks the flag before serving each request and rewrites to different pages. Flags update without redeployment. Show sub-5ms overhead by measuring middleware execution time. Add geolocation-based routing: serve a different page for users in India vs users in the US.

■ Auth.js (NextAuth) — Full Authentication

EASY

Add Google OAuth to a Next.js App Router project using Auth.js. Protect /dashboard with a session check. Show the session object on the protected page.

MEDIUM

Add credentials-based login alongside Google OAuth. Validate credentials against your database with bcrypt. Add the user's role to the session token (JWT callback). Protect routes based on role using middleware.

HARD

Implement a full enterprise auth setup: OAuth (Google, GitHub), credentials, email magic links, TOTP 2FA, and session expiry with refresh. Store sessions in a database (Auth.js adapter). Add an admin page that lists all active sessions and allows revoking individual ones. Security log every login, logout, and failed attempt with IP and user agent.

■ Next.js Deployment — Vercel and Self-Hosted

EASY

Deploy your Next.js app to Vercel. Configure environment variables for production. Set up preview deployments for pull requests. Verify the production build does not expose development secrets.

MEDIUM

Self-host the same Next.js app on a VPS (DigitalOcean or Hetzner). Run it in a Docker container behind Nginx with SSL from Let's Encrypt. Configure the Nginx reverse proxy to pass the correct headers (X-Forwarded-For, X-Real-IP) so Next.js sees the real client IP.

HARD

Set up a zero-downtime deployment pipeline: GitHub Actions builds the Docker image on push to main, pushes to a registry, SSHes into the VPS and runs a rolling deployment (start new container, health check, stop old container). The site must never return 502 during deployment. Verify with a continuous curl loop during deployment that shows no failed requests.

■ Linux — File System and Permissions

EASY

Navigate the file system: find all .log files modified in the last 7 days, show their sizes, and compress them into a tarball. Use find, stat, and tar. Do not use a GUI.

MEDIUM

Set up a Node.js application with correct Linux permissions: a dedicated non-root system user (nodeapp) owns the app files. The app directory is not world-readable. Logs go to /var/log/nodeapp with logrotate configured. The app runs as nodeapp via systemd and restarts on crash.

HARD

Diagnose a simulated production issue using only CLI tools: a process is consuming 100% CPU (simulate with a busy loop). Identify it using top/htop, find which port it is listening on with ss, inspect its open files with lsof, check its logs with journalctl, and gracefully restart it with systemctl. Document each command and what you learned from it.

■ Docker — Dockerfiles and Compose

EASY

Write a Dockerfile for your Node.js API. Build and run it. Verify the app works identically inside the container as outside. Use .dockerignore to exclude node_modules and .env.

MEDIUM

Build a multi-stage Dockerfile: stage 1 installs all dependencies and compiles TypeScript, stage 2 copies only the compiled output and production node_modules. The final image should contain no TypeScript, no devDependencies, and no source files. Compare the image sizes: naive vs multi-stage.

HARD

Write a docker-compose.yml for a full application stack: Next.js frontend, Node.js API, PostgreSQL, Redis, and Nginx. Services communicate by container name. Nginx is the only container with an exposed port. Database data persists via a named volume. The API only starts after PostgreSQL is healthy (healthcheck + depends_on condition). Document every line.

■ GitHub Actions — CI/CD Pipelines

EASY

Write a workflow that runs on every pull request: install dependencies, run TypeScript type check, run ESLint, run unit tests. If any step fails, the PR is blocked from merging.

MEDIUM

Build a full CI/CD pipeline: on push to main, run tests, build a Docker image tagged with the git SHA, push to Docker Hub (or GHCR), then SSH into a server and pull + run the new image. Use GitHub Secrets for credentials. The workflow must fail if tests fail.

HARD

Build a matrix workflow: test your Node.js app against Node 18, 20, and 22. Run the DB integration tests only against the current LTS. Add a dependency review step that fails if a PR adds a package with a known critical vulnerability. Implement `workflow_dispatch` so you can manually trigger a deployment to staging. Cache `node_modules` between runs and show the time improvement.

■ Nginx — Reverse Proxy, SSL, Rate Limiting

EASY

Configure Nginx to reverse proxy to your Node.js app on port 3000. Add gzip compression and set appropriate cache headers for static assets. Test with `curl -I` and verify the headers.

MEDIUM

Add SSL with Certbot (Let's Encrypt). Configure: redirect HTTP to HTTPS, HSTS header, modern TLS only (TLSv1.2+), strong cipher suites. Run your config through SSL Labs and achieve an A rating.

HARD

Configure Nginx as a load balancer for 3 upstream Node.js servers. Use `least_conn` algorithm. Add health checks that remove an upstream if it returns 5xx. Implement rate limiting at the Nginx level: 10 requests/second per IP with a burst of 20. Add a custom error page for 429 Too Many Requests. Test the failover: kill one upstream server and verify requests continue without errors.

■ Cloud — AWS/VPS Core Skills

EASY

Launch an EC2 instance (or VPS). SSH in with a key pair. Install Node.js. Deploy your app manually. Open the correct port in the security group. Access it in a browser.

MEDIUM

Set up S3 for file storage: create a bucket with appropriate permissions (not public), generate presigned upload URLs in your backend, upload a file using a presigned URL from the browser, generate a presigned download URL, and verify the file is not publicly accessible without the URL.

HARD

Build a complete production infrastructure: EC2 (or VPS) behind an Application Load Balancer, RDS PostgreSQL (or managed DB) in a private subnet, ElastiCache Redis (or managed Redis), S3 for file storage, CloudFront in front of S3 for CDN. Use IAM roles for the EC2 instance so no credentials are stored on the server. All resources in private subnets except the load balancer. Draw the architecture diagram before implementing.

■ Monitoring — Logs, Health Checks, Metrics

EASY

Add a `/health` endpoint to your API that returns: `{ status: 'ok', uptime, version, database: 'connected' | 'disconnected' }`. If the database is down, return 503 instead of 200.

MEDIUM

Set up structured logging with Pino: every request logs `{ method, url, statusCode, responseTimeMs, userId }`. Every error logs `{ message, stack, requestId }`. Use `pino-pretty` in development. In production, output newline-delimited JSON. Add request ID propagation so all logs from one request share the same `requestId`.

HARD

Build a metrics dashboard: instrument your app with Prometheus metrics (request count, latency histogram, active connections, error rate). Run Prometheus to scrape your metrics endpoint. Set up Grafana with dashboards for: p50/p95/p99 latency, requests per second, error rate, and memory usage. Configure an alert that fires when error rate exceeds 5%.

LAYER 7

■ SQL Injection

EASY

Write a deliberately vulnerable login endpoint that uses string concatenation in SQL. Exploit it with a classic ' OR '1'='1 payload. Then fix it with parameterized queries and show the exploit no longer works.

MEDIUM

Build a security test suite: write automated tests that attempt SQL injection against every endpoint that takes user input. Tests should pass (meaning the injections are blocked). Include: basic injection, UNION-based extraction, time-based blind injection payloads.

HARD

Set up a second-order SQL injection scenario: user input is stored safely but later used unsafely in a different query. Exploit it, then fix it by parameterizing the second query. Document why parameterization at storage time is not sufficient.

■ XSS — Stored, Reflected, and DOM-Based

EASY

Build a comment system vulnerable to stored XSS: store raw HTML from user input and render it. Exploit it with an alert-injecting payload. Fix it by escaping output.

MEDIUM

Implement a Content Security Policy header that blocks: inline scripts, eval, external scripts except from your own domain and a trusted CDN. Show that an XSS payload that worked before CSP is now blocked by the browser. Use report-uri to log CSP violations.

HARD

Find and fix DOM-based XSS: build a page that reads a value from the URL hash and inserts it into the DOM using innerHTML. Exploit it. Fix it using.textContent instead. Then audit your entire codebase for innerHTML, document.write, and eval usage and replace each with safe alternatives.

■ CSRF and Secure Cookies

EASY

Build a CSRF attack: create an attacker HTML page that submits a form to your app's API in the background. Show it works if cookies have no SameSite attribute. Fix it by adding SameSite=Strict.

MEDIUM

Implement a CSRF token system for forms that do not use SameSite cookies: generate a CSRF token per session, include it as a hidden field, validate it on every state-changing request. Show the attack fails with the token.

HARD

Conduct a full cookie security audit: every cookie your app sets must have: httpOnly (no JS access), Secure (HTTPS only), SameSite=Lax or Strict, appropriate Max-Age, and Path scoped correctly. Write automated tests that set each cookie and verify its attributes. Document why each attribute exists.

■ SSRF — Server Side Request Forgery

EASY

Build a vulnerable URL preview endpoint: it accepts a URL and fetches the page to show a preview. Show that an attacker can use it to fetch `http://169.254.169.254` (AWS metadata endpoint). Block it with an allowlist.

MEDIUM

Implement a robust SSRF defence: validate the URL is HTTP/HTTPS, resolve the hostname to an IP, block private IP ranges (10.x, 172.16-31.x, 192.168.x, 127.x, ::1), and block the cloud metadata IP (169.254.169.254). Show each bypass attempt fails.

HARD

Build a webhook delivery system (send HTTP requests to user-provided URLs) with SSRF protection. The protection must survive DNS rebinding attacks: validate the IP both before and after DNS resolution by binding to a specific resolved IP for the connection. Test with a DNS rebinding simulator.

■ Helmet.js, CORS, and Security Headers

EASY

Add helmet.js to your Express app. Run your app through securityheaders.io before and after. Document which headers helmet adds and what attack each prevents.

MEDIUM

Configure CORS correctly: allow only your frontend domain, specific HTTP methods, and specific headers. Show that a request from a different origin is blocked. Show that a same-origin request works. Handle preflight OPTIONS requests correctly.

HARD

Conduct a full security header audit. Achieve an A+ on securityheaders.io. This requires: Content-Security-Policy (no unsafe-inline), Strict-Transport-Security with preload, Permissions-Policy restricting camera/microphone/geolocation, Referrer-Policy, and X-Content-Type-Options. Document every header, what it prevents, and why your specific configuration is correct.

■ Dependency Auditing and Secrets Management

EASY

Run npm audit on a project. Fix all high and critical vulnerabilities. Add `npm audit --audit-level=high` to your CI pipeline so it fails on new critical vulnerabilities.

MEDIUM

Audit your codebase for leaked secrets: use truffleHog or git-secrets to scan the git history for accidentally committed API keys, passwords, or tokens. If found, rotate the secret, then purge it from git history using git filter-repo. Set up a pre-commit hook that blocks commits containing secrets.

HARD

Implement a secrets management system: no secrets in environment variables on the server. Instead, fetch secrets from AWS Secrets Manager (or HashiCorp Vault) at startup. Implement secret rotation: when a DB

password rotates, your app fetches the new secret without redeployment. Cache secrets in memory with a TTL. Handle the case where a secret fetch fails on startup gracefully.

■ Unit Testing with Jest / Vitest

EASY

Write unit tests for a pure utility module: a price calculator with discount logic. Cover: happy path, edge cases (0 quantity, 100% discount), and error cases (negative price). Achieve 100% branch coverage.

MEDIUM

Test a module with dependencies by mocking them. Your `OrderService` calls `PaymentService` and `EmailService`. In tests, mock both — `PaymentService` should succeed in happy path tests and fail in sad path tests. Never make a real payment or send a real email in unit tests.

HARD

Write a test suite for a complex state machine (order status transitions). Use `test.each` to run the same assertion logic across 20 different transition scenarios. Use `beforeEach` to reset state. Identify and test every impossible transition (should throw). Ensure tests run in under 100ms total — no `async`, no I/O.

■ Integration Testing with Supertest

EASY

Write integration tests for your `POST /users` and `GET /users/:id` endpoints. Use a real test database (separate from dev). Seed it before tests, clean up after. Test: success case, validation failure, not found.

MEDIUM

Test your authentication flow end-to-end with Supertest: register a user, log in, receive a JWT, use the JWT to access a protected route, use a wrong password and get 401, access a protected route without a token and get 401. All in one test file with a real database.

HARD

Write a full integration test suite for a feature (e.g. blog posts): CRUD operations, pagination, search, permission enforcement (admin can delete any post, user can only delete their own). Set up a test database with migrations in `beforeAll`. Each test should be independent — use database transactions that roll back after each test for speed, rather than truncating tables.

■ Component Testing with React Testing Library

EASY

Test a `LoginForm` component: fill in email and password, click submit, verify the `onSubmit` handler was called with the correct values. Test the validation: submit with empty fields, verify error messages appear.

MEDIUM

Test a data table component: mock the API call (use MSW — Mock Service Worker), verify the loading skeleton shows, then the data renders, then test filtering, sorting, and pagination by interacting with the UI (not by checking internal state).

HARD

Test an entire user flow in the component tree: render the app with a MemoryRouter at /login, fill in credentials, submit, verify redirect to /dashboard, verify the dashboard shows the logged-in user's name. Mock all API calls with MSW. Test the error flow: bad credentials show an error without redirect. The tests must query the DOM the way a user would (getByRole, getByLabelText) — no test IDs except as a last resort.

■ E2E Testing with Playwright

EASY

Write a Playwright test that: opens the login page, fills in credentials, submits the form, and asserts the dashboard URL and a welcome message. Run it in headless Chrome.

MEDIUM

Write an E2E test for the full user registration flow: open /register, fill all fields, submit, verify redirect to /dashboard, log out, log back in with the new credentials. Add a visual screenshot assertion at the dashboard page.

HARD

Build a full Playwright test suite covering 5 critical user flows. Set up: parallel test execution, a global setup that seeds the test database, API mocking with Playwright's route interception for external services, and automatic screenshots + video recording on failure. Run the suite in your GitHub Actions CI on every pull request. The full suite should complete in under 2 minutes using parallelism.

■ Test Doubles — Mocks, Stubs, Spies

EASY

Use `jest.fn()` to spy on a function: verify it was called, how many times, and with what arguments. Use `jest.spyOn` to spy on a method of an existing object without replacing it.

MEDIUM

Build a test for a function that calls an external API. Implement three versions: (1) mock the entire module, (2) stub the HTTP layer with `nock`, (3) use `MSW` to intercept at the network level. Explain which approach gives the most confidence and why.

HARD

Implement the test doubles yourself without Jest: write `stub(fn, returnValue)`, `spy(obj, method)` that records calls, and `mock(obj, method, implementation)` that replaces the method and records calls. Each must have a `restore()` function. Use them to test a class with 3 collaborators without mocking libraries.

■ Horizontal vs Vertical Scaling + Stateless Servers

EASY

Draw a diagram showing the same app scaled vertically (bigger server) vs horizontally (multiple servers + load balancer). List 3 limits you hit with vertical scaling that horizontal solves, and 3 new problems horizontal introduces.

MEDIUM

Make your Express app stateless: move sessions from in-process memory to Redis. Start two instances of the app on different ports, load balance between them with Nginx. Verify a session created against instance 1 works against instance 2.

HARD

Design a stateless deployment where your app can scale from 1 to 100 instances with no code change. Everything that is per-request-stateful (sessions, rate limit counters, job locks, websocket state) must live in an external store. Write a checklist of everything you moved out of process memory and where it now lives. Demonstrate with a 10-instance docker-compose setup.

■ Load Balancing Algorithms

EASY

Configure Nginx with round-robin load balancing across 3 upstream servers. Add a custom header X-Server-ID to each server's response. Make 30 requests and verify they distribute roughly evenly.

MEDIUM

Compare least_conn vs round-robin: create 3 upstreams where one is artificially slow (sleep 2 seconds). Show that round-robin sends traffic to the slow server equally, while least_conn routes around it. Measure total request throughput for both.

HARD

Design and explain consistent hashing: draw a hash ring, show how servers are placed, show how a key is mapped to a server, and show that adding a server only remaps 1/N of keys (not all of them). Implement a basic consistent hash ring in Node.js. Show it distributes 10,000 keys roughly evenly and that adding a server remaps only ~25% of keys.

■ Caching Strategies

EASY

Implement cache-aside for a user profile: check Redis first, on miss fetch from DB and populate cache. Show cache hit and miss in logs. Add a 5-minute TTL.

MEDIUM

Implement write-through caching: on every DB write, also update the cache. Compare to cache-aside for a write-heavy workload. Show where write-through wastes resources (writing to cache for data that is never

read).

HARD

Design a cache invalidation strategy for a social feed: a user's feed is cached. When a followed user posts, all followers' feed caches must be invalidated. With 1M users each following 500 others, naive invalidation is too slow. Design a fanout-on-write vs fanout-on-read tradeoff analysis. Implement one approach and document when you would choose the other.

■ Message Queues — Kafka vs RabbitMQ Concepts

EASY

Draw diagrams showing how RabbitMQ (broker with queues and exchanges) and Kafka (log-based, partitioned, consumer groups) differ architecturally. List 3 use cases better suited to each.

MEDIUM

Implement a publish-subscribe system using BullMQ: when an order is placed, publish an OrderPlaced event. Three subscribers process it independently: EmailService, InventoryService, and AnalyticsService. Show all three fire for each event.

HARD

Design a system that processes 100,000 orders per minute reliably. Draw the architecture: producers, topics/queues, consumer groups, dead-letter handling, and the downstream databases. Answer: how does the system handle a consumer being down for 2 hours and then coming back up? How do you ensure exactly-once processing for payments? What happens if the queue itself goes down? Write the design document.

■ Reliability — Circuit Breaker, Retry, Graceful Degradation

EASY

Implement retry with exponential backoff for an HTTP client: on 5xx or network error, retry up to 3 times with delays of 1s, 2s, 4s. Add jitter (random 0-100ms) to each delay. Log each retry attempt.

MEDIUM

Implement a circuit breaker: after 5 consecutive failures, open the circuit (fail fast without calling the downstream service). After 30 seconds, half-open (allow one test request). If it succeeds, close the circuit. If it fails, reopen. Show the state transitions with a failing dependency.

HARD

Design and implement graceful degradation for your app: if Redis is down, the app falls back to no caching (slower but functional). If the email service is down, queue emails for later rather than failing the request. If the database read replica is down, fall back to the primary. Implement health checks that report degraded (not down) status when running in fallback mode.

■ System Design Problem — URL Shortener

EASY

Design a URL shortener on a whiteboard: describe the data model, the shortening algorithm, and the redirect flow. Estimate storage needs for 100M URLs.

MEDIUM

Implement a working URL shortener: generate a 6-character base62 short code, store in PostgreSQL, redirect on access, track click count. Handle the case where a short code collides.

HARD

Scale the URL shortener to handle 100,000 redirects per second: add Redis caching for hot URLs, design the read path to never hit the database for cached URLs, implement rate limiting on URL creation, add analytics (count clicks by country, device, referrer) without slowing down the redirect (use async event processing). Draw the full architecture and explain every component's role.

■ System Design Problem — Notification System

EASY

Design a notification system on a whiteboard: what inputs trigger notifications, what delivery channels exist (push, email, SMS, in-app), how do you fan out to 1M subscribers.

MEDIUM

Implement an in-app notification system: create notifications via API, deliver in real time to connected users via SSE, persist for offline users, mark as read. A user with 3 browser tabs open receives the notification in all tabs.

HARD

Design a notification system that sends 10M push notifications in under 5 minutes. Address: batching, priority queues (transactional vs marketing), per-user preferences (opt-out per channel), rate limiting per provider (APNs throttles), and failure handling (retry with exponential backoff, dead-letter queue for undeliverable). Write a design document with a bottleneck analysis.

■ System Design Problem — Rate Limiter

EASY

Explain four rate limiting algorithms: fixed window, sliding window log, sliding window counter, and token bucket. For each: draw a diagram, explain time complexity, and identify the edge case it handles or introduces.

MEDIUM

Implement a sliding window rate limiter in Redis using a Sorted Set. Wrap it in a Lua script for atomicity. Apply it as Express middleware with per-user limits. Test with artillery: send 200 requests in 10 seconds from the same user (limit: 100/min) and verify exactly 100 succeed.

HARD

Design a distributed rate limiter for a multi-region API (US, EU, Asia). The limit is global (a user cannot use 100 requests in US and another 100 in EU). Discuss tradeoffs: strong consistency (synchronous cross-region coordination) vs eventual consistency (each region enforces locally with periodic sync). Implement the eventual consistency approach and show where it over-allows vs the ideal.

LAYER 10

This is what makes you sound senior to a CTO. Think in tradeoffs, not absolutes.

■ Layered Architecture — Routes, Controllers, Services, Repositories

EASY

Refactor a spaghetti Express app (all logic in route handlers) into 4 clean layers. No database query may appear outside a repository. No business logic in routes. Draw the dependency graph before and after.

MEDIUM

Enforce the layer architecture with tests: write a test that imports all service files and asserts they do not import from any route file. Write another that imports all repository files and asserts they only import from database libraries (not services). These tests run in CI and fail on violations.

HARD

Build the same feature (user registration with email verification) in two architectures: layered monolith and a clean architecture (ports and adapters / hexagonal). In the hexagonal version, the business logic (domain) has zero dependencies on Express, PostgreSQL, or any framework. Swap the database from PostgreSQL to an in-memory store without changing one line of business logic.

■ SOLID Principles — Applied, Not Memorised

EASY

Find a violation of each SOLID principle in a codebase (write the violations yourself). Fix each violation. Name the principle violated and why the fix is correct.

MEDIUM

Refactor a UserService that handles registration, login, profile update, email sending, and avatar upload. Apply Single Responsibility: split it into focused classes. Apply Dependency Inversion: depend on interfaces (TypeScript interfaces), not concrete implementations. Show that you can swap the email provider without touching UserService.

HARD

Design a payment processing module that supports Stripe, Razorpay, and PayPal using the Open/Closed principle: adding a new provider requires only a new class, no changes to existing code. Use the Strategy pattern + Dependency Injection. Write tests that run against a mock provider to verify the abstraction holds.

■ Monolith vs Microservices Decision

EASY

List 5 real reasons to choose a monolith first and 5 genuine signals that a specific part of a monolith should become a service. Argue both sides as if presenting to a CTO.

MEDIUM

Take your monolith blog app and extract the notification system as a separate service that communicates via HTTP. The monolith calls the notification service when events occur. Document: what new problems did extraction create (distributed tracing, network failures, versioning the API between services).

HARD

Design a migration plan from a monolith to microservices using the Strangler Fig pattern: new features go in new services, old features are extracted one at a time behind an API gateway. The monolith continues to run throughout. Address: how do you handle data that is shared between the monolith and the new service? How do you handle transactions that span both? Write a 2-page design document.

■ **REST vs GraphQL — Architecture Decision**

EASY

Build the same API twice: a REST version and a GraphQL version (use Apollo Server or Pothos). Compare: client request count for a dashboard page (REST probably needs 3 requests, GraphQL needs 1), ease of adding a new field, and caching complexity.

MEDIUM

Solve the N+1 problem in GraphQL using DataLoader: a query for 10 posts with their authors should trigger exactly 2 DB queries (one for posts, one for all authors batched). Show the query logs before and after DataLoader.

HARD

Design the right API strategy for a product with: a public REST API (third-party developers), a mobile app (needs efficient data fetching), and a web dashboard (complex, nested data). Argue why GraphQL for the web/mobile client and REST for the public API is correct. Design the gateway layer that serves both from the same underlying services.

■ **Making Architecture Decisions — The Framework**

EASY

Take any architectural decision from your current project (or invent one: 'should user sessions be stored in cookies or localStorage?'). Write a decision document: problem statement, requirements, constraints, options considered, tradeoffs of each, decision, and consequences.

MEDIUM

Write three Architecture Decision Records (ADRs) for your ambitious project. Each must follow the format: title, status, context, decision, consequences. The decisions should be non-trivial: database choice, caching strategy, authentication approach.

HARD

Conduct a full architecture review of your most complete project. Produce a document covering: current architecture diagram, what works well and why, three architectural weaknesses with business risk (not just technical complaints), and a prioritised remediation plan. Present this document as if to a CTO hiring you to fix their system. This is the exercise that prepares you most directly for senior remote interviews.

■ **DSA — Interview Preparation**

EASY

Solve 20 LeetCode Easy problems across: arrays, strings, hashmaps, and two pointers. For each, write the solution in TypeScript and articulate the time and space complexity before submitting.

MEDIUM

Solve 20 LeetCode Medium problems across: sliding window, binary search, basic trees (BFS/DFS), and recursion. For each medium problem, write your brute force first, identify the bottleneck, then optimise. Document the optimisation insight.

HARD

Solve 5 system-design-adjacent LeetCode problems: LRU Cache, Design Twitter (simplified), Design a Hit Counter, Rate Limiter, and a Task Scheduler. These bridge pure algorithms and real system design. For each, explain not just the code but how you would extend it to a real distributed system.

You now have the map.

The assignments are designed to be done in order, but treated as real deliverables — not toy exercises. Ship the code. Break it. Read the error. Fix it. That loop, repeated a few hundred times across these layers, is the entire curriculum.

Full-time pace: 5–6 months to completion. International remote offer ready.